4G RTOS & Android Watches Integraltion Protocol

Update log

Date	Update the content	Regenerator
2022-04-19	Resign the agreement	ZYH
2022-04-22	Add the instruction to upload motion data Supplement the instruction to upload health data and set the measurement frequency of the device	ZYH
2022-04-25	Add the instruction identifier for the public request body, message notification instruction, and custom extension instruction (upload and download)	ZYH
2022-05-23	Login, upload sleep data, upload exercise data, set device configuration, upload health data adjustment Added OTA upgrade instructions	ZYH
2022-05-24	Supplementary agreement-unified description	ZYH
2022-05-31	Description of usage scenarios of supplementary agreement, etc	ZWC
2022-06-07	Adjust protocol fields: 1. Upload the activity data of today and modify the exerciseTime field attribute 2. Upload sleep data command parameter adjustment 3. Split the SOS text message switch and the carousel display frequency settings in the device configuration instructions 4. Modify the type of custom extension instructions to distinguish between uplink and downlink	ZWC
2022-06-14	The ref parameter for the public request/response body is modified to be compatible	ZWC
2022-06-28	Add parameters to the instruction for issuing OTA information	ZWC

Date	Update the content	Regenerator
2022-07-06	Set the parameters of the measurement frequency to correct the parameters (note that the unit is minutes)	ZWC
2022-07-12	Add TCP mode docking	ZYH
2022-07-18	Increase 1. Request to issue default device configuration 2. The device actively restores factory Settings	ZWC
2022-07-22	1. Modification: The device actively restores factory Settings 2. Login instruction Reply Add binding identifier: bindStatus 3. Supplementary explanation of the ident of the public response structure 4. Add the instruction "issue watchbound status" dnDevBindStatus	ZWC
2022-08-03	1. Add parameters and instructions for obtaining and issuing OTA series commands	ZWC
2022-08-29	1. Add the parameter useDataFlow to the OTA configuration instruction	ZWC
2022-10-10	1. The format of ECG, HRV, PPG and RR for the upload of health data was modified and instructions were added	ZWC
2022-10-27	1. Add parameters and explanations for the "request to measure health data" and "health data upload" instructions	ZWC
2022-12-22	1. Add optional parameters (chip information, firmware version, etc.) uploaded by the device 2. Login instruction Add the parameter dateProduction: The production date of the equipment	ZWC
2023-02-03	1. Add optional parameters such as cpuModel to the login instruction	ZWC
2023-02-13	1. Add optional parameter deviceIP to the login instruction	HSG
2023-03-31	1. The login instruction adds the function configuration of uploading devices	ZYH
2023-06-12	1. Add a series of video call instructions	ZWC
2023-06-15	Video call： 1. The upWatchHangUp command removes the vendor field 2. APP/Device Invite device video/voice call- Change the instruction type to downVideo Call With Watch 3. Adjust the instruction down APP Hang Up description information	QZC
2023-06-20	1. Enter the platform field for device login	QZC

Date	Update the content	Regenerator
2023-06-29	1. Add a protocol for uploading sensor values 2. Adjustment and optimization description	ZWC
2023-07-04	1. Add a batch data upload protocol	HSG
2023-08-16	1. Add the instruction to report device call records 2. Add a series of micro chat instructions and HTTP interactive interface	QZC
2023-08-24	1. Report the call record and supplement the connection type field and wait for the connection duration field	QZC
2023-08-28	1. Adjust the instruction for batch uploading of sensor values to the device. Add dataTime to the array of data measurement time	HSG
2023-09-05	1. Add the optional parameter key to the login instruction 2. Set the alarmClock parameter status type to string 3. Add ECG analysis data uploaded 4. Send ECG analysis results 5. Add the weather field to the weatherType field	HSG
2023-09-19	1.Query the latest sleep command 2. Send sleep data	HSG
2023-09-26	1. Add the coordinate system type field to upload GPS data	QZC
2023-11-07	1. Send sleep data command and add upDayStr field	QZC
2023-11-27	1. Adjust the text description uploaded by upBatch command	QZC
2023-11-28	1. Add the device reporting function configuration instruction-up Device Config	QZC
2023-12-01	Increase the respiratory rate	txj
2023-12-19	1. The device actively obtains the device binding status-upGetDevBindStatus	QZC
2024-01-22	1. Upload the motion data and <upRun> supplement the optional values of motion type	txj
2024-03-06	1. Add dial switch <dialSwitch> function configuration	QZC
2024-04-10	1. Add the type of exercise data <upRun>to be uploaded 2. Add a medication plan instruction <dnMedicationPlan>	txj
2024-05-31	1. Family number <familyNumber> is assigned to supplement the international area code prefix field	QZC

Date	Update the content	Regenerator
2024-06-25	Device configuration: New configuration (PPG blood pressure trend, heart blood management health risk warning, etc.), high heart rate reminder (new exercise heart rate), sleep setting (Adjust the format of the sleep target duration field)	TXJ
2024-07-10	1. Add a no-disturb command- <disturb>	QZC
2024-10-14	New child features: 1. Remote photography 2. Prohibition of class attendance 3. Meet people 4. Application control 5. Automatic answer 6. Set up a red flower 7. Set up the device configuration to add-remove the password	QZC
2024-12-06	1. Prohibit the distribution of instructions in class, and adjust application control and report instructions	QZC
2025-06-16	Add a new schedule, remind you to drink water and sit for a long time, set device configuration (volume setting, automatic wrist lift switch, 12/24 time system, power saving mode, device language, device loss), dial, Astrology	TXJ
2025-06-17	1. Add video (visual communication) parameters directly with WeChat Mini Program (upWatchHangUp; upGetVideoUserList downVideoCallWithWatch)	ZWC

Agreement Summary Statement

Contact the backend developer for specific domain names and ports

======= Below are the necessary operations for the device docking platform =======

The current approach is to use the domain name resolution into IP and then use IP+ port to connect to the background server.

The following connection process is required for the docking equipment:

Each time the domain name is resolved into an IP address for connection, the IP address of the current domain name resolution needs to be saved.

When the domain name is unavailable **(DNS cannot resolve to IP, or the resolved IP connection fails repeatedly (after 10-20 reconnections)), use IP to connect (no retry failures)**

==========

TCP/IP interface

```
-----------------------------------------------------------------------
| Type(2 byte) | Length(2 byte) |  json original data
-------------------------------------------------------
```

.Each instruction contains a 4-byte header field,
.The first two bytes are the start field, which is a large-endian unsigned short integer with the identifier: FCAF
.The last 2 bytes are the length identifier, which is a large-endian unsigned short integer indicating the length of the data following it.
.The size of the entire instruction data packet is 4 + Length (the value of the instruction header length field);

Example command (device login command):

```
\xFC\xAF\x01\xBA{"type":"login","ident":762250,"ref":"w:update","imei":"666
```

Note: "XUFA01BA" is only a byte type hexadecimal string display. Since byte cannot be displayed directly, it is shown like this. When transmitting instructions, note that the header is 4 byte data, not these strings.

Connect Ways via **Websocket**

The device data information is connected in a unified manner through Websocket;

Content is transmitted in JSON format;

In the following protocol, the parameters with an asterisk are optional and not mandatory

eg:

```
"extend": null [*] // indicates that this parameter is not mandatory
 perhaps
"extended" *: null // indicates that this parameter is not mandatory
```

In the following public response request:

Ident is the identifier of the instruction. A pair of instructions is issued/replied with the same **ident** identifier, indicating that this instruction is a reply to a specific instruction.

For example, if the id of the device upload login command is **123456**, then the server will reply with the same ID, which indicates a pair.

Device uploads data/replies to commands Common request body:

```
{
    "type": "xxx",
    "ident": 762250,
    "ref": "w:update",
    "imei": "xxxxxx",
    "extend": null, [*]
    "data": {
        "type": "xxx",
        "imei": "xxxxxx",
        "deviceModel": "xxx",
        "timestamp": 1648111390074,
        ...
    },
    "timestamp": 1616664712235
}
```

field	Is it necessary ?	Explanation	remarks
type	yes	type of instruction	If there is no special explanation, two types are the same
ident	yes	Instructional identification	[int] Instruction flag (6-digit random number): The receipt and dispatch are the same instruction identifier, Indicates a pair
ref	yes	source	Device： w: reply w: update
imei	yes	Device imei number	
timestamp	yes	time stamp	Millisecond timestamp
data	yes	concrete content	Refer to the device upload instruction
Device Model	yes	Device Model	
extend	deny	Expand fields	json content

Server responds to device / issues instructions - Public response body:

```
{
    "type": "xxx",
    "ident": 762250,
    "ref": "s:reply",
    "imei": "xxxxxx",
    "data": {
        "type": "xxx",
        "imei": "xxxxxx",
        "timestamp": 1648111390074,
        ...
    },
    "extend": null, [*]
    "timestamp": 1616664712235
}
```

field	It has to be	Explanation	remarks
type	yes	type of instruction	If there is no special explanation, the type in data is the same
ident	yes	Instructional identification	[int] Instruction ID (6-digit random number): The receive and send are the same instruction ID, Indicates a pair
ref	yes	source	server： s: reply. s: down.
timestamp	yes	time stamp	Millisecond timestamp
data	yes	concrete content	json content
imei	yes	Device imei number	
extend	deny	Expand fields	json content

Upload instructions to the device

Special explanation：

About the reporting of data recorded and measured by the equipment:

Health，

Sleep ,

Today's activity (keep the

latest one),

Exercise,

Locationn,

SMS reporting,

Remote pickup

Success: The device can only consider the data upload successful when the data is reported successfully and the server responds to the corresponding instruction successfully.

Failure: When the network/server causes the failure to receive corresponding response instructions, local caching should be implemented. A retransmission mechanism should then be established to resend after a certain period of time.

eg: Cache data and retransmit after 5-15 minutes. (Note!: The storage time of data is recorded at the time of recording.)

Custom extension instruction (upload) -upCustom

Explanation:

Subsequent instructions that can be expanded at will have no fixed purpose and will retain the corresponding connection instructions.

```
{
    "type": "upCustom",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    ... Custom parameters
    "timestamp": 1648111390074
}
```

Login-

Explanation:

After the device and the background establish the WebSocket, the first command to be uploaded should be the login command, and then the subsequent commands can be effective.

```
{
    "type": "login",
        "imei": "865028000000306"
                                ,
```

```
    "deviceModel": "xxx",
    "platform": "ASR", // platform: ASR Android
    "iccId": "89860121801575995911", [str] // The ICCID of the device SIM card
    "Phone*:" 17612002284", [str] // The number of the SIM card in the device
    "batteryLevel": 64, [int] // Power
    "Version": "1.2.42", [str] // Firmware version
    "mac*:" 8c:14: b4:42:44:00", [str] //MAC address The real MAC address of the device
    "SN" *: "*****", // The real SN of the device
    "OS" *: "ThreadX", // The operating system, android 9.0, Linux 2.6.18 and other operating
    system versions are required
     "firmware_v"*: "*****", // Firmware version: 22.2.6202
     software_v"*:"*****", // Software version (including version name and version number).
     Multiple versions can be separated by semicolon
     "dateProduction" *: "2022-12-22", //Device production date
     "chips" *:[//Chip information, can contain multiple groups eg:ASR3603
       {
     "type": "LTE_CAT1" // Chip type, Main/WiFi/Zigbee/BLE, etc. Must be filled.
     Eg: LTE_C "Factory": "ASR" // Chip manufacturer Eg: ASR
     "model": "3603" // Chip model: eg: 3603}
       ...  //  There may be multiple chip information
    ],
    "cpuModel"*:                    "****",   // Application processor (AP) model,which must be filled in
    "romStorageSize"*:              "****",   when the device is in use
    "ramStorageSize"*:              "****",   // The total storage capacity (ROM) of the device, and the total memory capacity
    "networkType"*:                 "****",   (RAM) of the device, which are registered with the Ministry of Industry and
    "deviceVendor"*:                "****",   Information Technology.
    "deviceBrand"*:                 "****",   // The current network type used by the device, such as RJ45 (wired), Wi-Fi,
    "deviceModel"*:                 "****",   //Device manufacturer
    "deviceIP"*:                    "****",   //Device brand
    "powerSupplyMode"*:             "****",   // Device model
                                              // Device IP
                                              // Power supply mode: Battery power supply: battery, POE power supply:

    "config*": {}, //[obj] device function configuration
    "key" *: "****",//device secret key
    "timestamp": 1648111390074
}
```

Description of device function configuration:
Id: Corresponding function id is default
PID: Corresponding function PID
Default name: Function name
Key: Function key, corresponding function key as default
Type: Function type. If yes, the corresponding function type is default
url: Function corresponding to the url. If there is, the corresponding function url defaults (except Device instruction device usage instructions).
belong: Function corresponding to belong defaults
Children: Subfunction
For Example:

```
"config": {

        "Index": [// [array] homepage function configuration, the value can be:
      blood pressure, heart rate, blood oxygen saturation, body temperature, step,
      "Blood pressure",
      " Heart rate",
      "Blood oxygen saturation",
      "Steps"
        ],
      deviceFunction: [=[array] Device function page configuration, which can be customized in
            different regions. {=[obj] Region 1
                  "region": [
                        {
```

```
        "id": "5ea02f20-7654-4989-9628-84121f1b7777"  "name": "About device",//about device
        "key": "aboutDevice",
        "type": "url",
        "url": "details",
        "belong": 2
    },
    {
        "id": "5ea02f20-7654-4989-9628-84121f1b8888",
        "name": "Device instruction", //device usage instructions
        "key": "deviceExplain",
        "type": "url",
        "url: "https://www.baidu.com/", // Use instructions external link
        "belong": 2
    },
    {
        "id": "324469f5-9ae7-4df7-8358-eebf06f76e16",
        "name": "Health monitoring", // health monitoring
        "key": "healthMonitoring",
        "type": "url",
        "url": "function/healthTest",
        "belong": 2,
        "children": [
            {
                "id": "0c6b9cd6-9052-4620-953b-1f72bc5155cd
                "name": "Target steps", // Target steps
                "key": "StepTarget",
                "belong": 2,
                "pid": "324469f5-9ae7-4df7-8358-eebf06f76e1
                "defaultValue": "6000" // Default value
            },
            {
                "id": "251ded48-5655-4932-a353-069948d1186e
                "name": "Sleep Settings", // sleep Settings
                "key": "SleepIntervalOrSwitch",
                "type": "none",
                "belong": 2,
                "pid": "324469f5-9ae7-4df7-8358-eebf06f76e1
                "default value": 1, // 1/0 to enable or disable
                "sleepStartTime": "220000", // sleep start time,
                "sleepEndTime": "090000" // Sleep end time
            },
            {
                'id':"574e5610-4641-4611-bb66-6a9084f24f5c",
                "name":"heart rate detection",//automatic heart rate detection,
                "key": "upHeartRate",
                "belong": 2,
                "pid": "324469f5-9ae7-4df7-8358-eebf06f76e1
                "explain": "xxxxxx", // function item description, displayed on the page
                "defaultLabel": "30minutes/time", // currently selected
                "defaultIndex": 0, // currently selected index of option
                "styleType": 1, // Page style type, 1= single selection list (new page
                "defaultValue": "30", // // Currently selected option's
                "option": [// Option configuration, here configure the number of items on the page
                    {
```

```
                "showText": "30minutes/time",
                "communicatekey": "30"
            },
            {
                "showText": "60minutes/time", // selected
                "communicatekey": "60", // selected
                "checked": false // If not, it is the current selection
            }
        ]
    },
    {
        'id': "25c533b1-3db4-441c-ae13-700bbb8f3dd6",
        name: "Blood pressure detection", // blood pressure
        "key": "upBP",
        "belong": 2,
        "pid": "324469f5-9ae7-4df7-8358-eebf06f76e1
        "explain": "xxxxxx",
        "defaultLabel": "30minutes/time",
        "defaultIndex": 0,
        "styleType": 1,
        "defaultValue": "30",
        "option": [
            {
                "showText": "30minutes/time",
                "communicatekey": "30"
            }
        ]
    },
    {
        "Id": "1f9dbcb0-1e85-4e6c-a27a-19ee0f21c2cb",
        "name": "Blood oxygen detection", // Blood oxygen automatic
        "key": "upBO",
        "belong": 2,
        "pid": "324469f5-9ae7-4df7-8358-eebf06f76e1
        "explain": "xxxxxx",
        "defaultLabel": "30minutes/time",
        "defaultIndex": 0,
        "styleType": 1,
        "defaultValue": "30",
        "option": [
            {
                "showText": "30minutes/time",
                "communicatekey": "30"
            }
        ]
    },
    {
        "id": "2a710af5-9ca2-4bcd-99f9-d0f7b45ec4c1",
        "name": "Temperature monitoring", //temperature monitoring
        "key": "upBodyTemperature",
        "belong": 2,
        "pid": "324469f5-9ae7-4df7-8358-eebf06f76e1
        "explain": "xxxxxx",
        "defaultLabel": "30minutes/time",
        "defaultIndex": 0,
```

```
                "styleType": 1,
                "defaultValue": "30",
                "option": [
                    {
                        "showText": "30minutes/time",
                        "communicatekey": "30"
                    }
                ]
            },
            {
                "id": "990ddfdf-fae3-4921-bcd1-cebf433c99ae
                "name": "Step synchronization", // step synchronization
                "key": "upStep",
                "belong": 2,
                "pid": "324469f5-9ae7-4df7-8358-eebf06f76e1
                "explain": "xxxxxx",
                "defaultLabel": "30minutes/time",
                "defaultIndex": 0,
                "styleType": 0,
                "defaultValue": "30",
                "option": [
                    {
                        "showText": "30minutes/time",
                        "communicatekey": "30"
                    }
                ]
        },
        {
            "id": "a86abd91-0e5e-4b34-b8a2-6cd97399ee12",
            "name": "Telephone directory", // telephone book
            "key": "telephoneDirectory",
            "type": "url",
            "url": "function/phones",
            "belong": 2,
            "value": "10" // Quantity limit
        },
        {
            "id": "0e6d0ca7-9bfe-4946-941f-e460149fe5b6",
            "name": "SOS number", //SOS number
            "key": "SOSNumber",
            "type": "url",
            "url": "function/sosList",
            "belong": 2,
            "Value": "10" // Quantity limit
        },
        {
            "id": "7c844cce-1bbb-41ef-afa8-3e90242f1de4",
            "name": "SOS Settings", //SOS Settings
            "key": "SOSSetting",
            "type": "url",
            "url": "function/sos",
            "belong": 2,
            "children": [
```

```
                {
                    'id': "6859476e-79b5-4949-a94a-65b792c67da9",
                    "name": "Trigger SOS to send SMS", // trigger S
                    "key": "SOSSwitch",
                    "belong": 2,
                    "pid": "7c844cce-1bbb-41ef-afa8-3e90242f1de
                    " default value: 1 // 1/0 On/off
                },
                {
                    "id": "aeb33177-ad95-48f5-9653-b455d8356039
                    "name": "SOS rotation number", //SOS wheel
                    "key": "SOSRotation",
                    "belong": 2,
                    "pid": "7c844cce-1bbb-41ef-afa8-3e90242f1de
                    "explain": "xxxxxx",
                    "defaultLabel": "3times", // 3 times
                    "defaultIndex": 0,
                    "styleType": 1,
                    "defaultValue": "3",
                    "option": [
                        {
                            "showText": "3time",
                            "communicatekey": "3"
                        }
                    ]
                }
            ]
        },
        {
            "id": "5ea02f20-7654-4989-9628-84121f1b666d",
            "name": Device call charge, // Device call charge
            "key": "telephoneCharge",
            "type": "url",
            "url": "function/cost",
            "belong": 2
        },
        {
            "id": "f297542b-c34f-4a35-80f2-79031afb02b3",
            "name:"  "Reject strangers", // reject strangers
            "key": "CallInLimitSwitch",
            "type": "switch",
            "belong": 2,
            "default value": 1 // 1/0 to enable or disable
        },
        {
            "id": "1e7609fc-3028-44c1-9ecf-c4a67400f816",
            "name": "Fall alarm", // fall alarm
            "key": "FallWarnSwitch",
            "type": "switch",
            "belong": 2,
            "default value": 1 // 1/0 to enable or disable
        },
        {
            "id": "1a547c79-9490-4d1b-865f-846dff6d03aa",
            "name": "Restart the device", // Restart the device
```

```
                "key": "restartDevice",
                "type": "confirm",
                "belong": 2
            },
            {
                "id": "f6703d33-d28c-4424-bce6-1586bdddabdd",
                "name":Remote shutdown"
                "key": "remoteShutdown",
                "type": "confirm",
                "belong": 2
            },
            {
                "id": "b06cdae1-b3df-4fa1-b151-7b758e293593",
                "name": "Factory data reset", // Restore factory Settings
                "key": "factoryReset",
                "type": "confirm",
                "belong": 2
            },
            {
                "id": "0e6d0ca7-9bfe-4946-941f-e460149fe5b7",
                "name": "Alarm clock" // Alarm clock (with audio)
                "key": "clock",
                "type": "url",
                "url": "function/clock",
                "belong": 2,
                "value": "3" // Quantity is online
            },
            {
                "id": "f297542b-c34f-4a35-80f2-79031afb02b99",
                "name": "Dial", // Dial Panel
                "key": "dialSwitch",
                "type": "switch",
                "belong": 2,
                "defaultvalue": 1 //  1/0 to enable or disable
            },
            {
                "id": "0e6d0ca7-9bfe-4946-941f-e460149fe5b8",
                "name": "Alarm clock", //Alarm clock (no audio)
                "key": "clock1",
                "type": "url",
                "url": "function/clock",
                "belong": 2,
                "value": "3" // Quantity limit
            },
            {
                "id": "5ea02f20-7654-4989-9628-84121f1b666d1",
                "name": "Device firmware upgrade", //device firmware upgrade
                "key": "upgrade",
                "type": "url",
                "url": "function/upgrade",
                "belong": 2,
                "default value": 0 //0=Automatic flow update is enabled 1
                =Automatic flow update is turned off
            }
        ]
    },
```

```
                {} //[obj] Region 2
                ...
            ]
        },
```

Login to reply

```
{
    "type": "login",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "bindStatus": 0, [int] Whether it is bound status 0: not bound 1: bound
    "timestamp": 1648111390074
}
```

Heartbeat

Explanation：

Send a heartbeat command at regular intervals from the server.

```
{
    "type": "heartbeat",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "batteryLevel": 100, [int][*] // Power
    "batteryState": 1, [int][*] // Charging status (0: not charged 1: charging)
    "timestamp": 1648111390074
}
```

Upload health data

Explanation：

1. The equipment reports health data regularly according to the set measurement frequency
2. The device performs a measurement and reports the health data instruction according to the request received from the server

```
{
    key
```

"imei": "865028000000306",
"deviceModel": "xxx",
"testType": 1, [int][*] // Measurement type (0: timed measurement 1: hardware manual measurement 2: app command measurement
"WearPosition" *: 1 [int] //Wear position of the watch (1=left hand, 0=right hand) (optional parameter)
"airbagSize" *: 1 [int] // airbag size (1= small, 2= medium, 3= large) (optional parameter)
"timestamp": 1648111390074
}

Key Explanation: Different health data types Key are the following fields heart rate
"type": "upHeartRate",
"date": *"100", [str] // heart rate value*
==========
//— Common parameters for the following four types of instructions:
"collectionLogo": "87654321", [str] // Single sampling identifier
Data start time: 1653228000000, [long] // Sampling start timestamp (millisecond level)
Data Status: 0, [int] // This packet data status 0: first packet data 1: intermediate packet data 2: end packet data (e.g. "block": 0 [int] // used to indicate which data is the first

ecg
"type": "upECG",
"date": "110.2, 120.6, 99.7", [str] // ECG data values (up to 500 sets) Frequency: "200", [str] // Sampling frequency (Hz) (500Hz ~ 800Hz)

HRV heart rate variability
"type": "upHRV",
"date": *"141", [str] //hrv, heart rate variability*
"Frequency": "200", [str] // Sampling frequency Hz (25HZ ~ 100HZ)

ppg photoelectric capacitive volume pulse wave recording
"type": "upPPG",
"date": "110.2, 120.6, 99.7", [str] //ppg (up to 500 groups)
"Frequency": "200", [str] // Sampling frequency Hz (40HZ ~ 220HZ)

RRI ventricular beat interval
"type": "upRR",
"data": "1648111390074,1080;1648111391154,1570;1648111392724,870", [str] /
"Frequency": "200", [str] // Sampling frequency Hz (40HZ ~ 220HZ)
==========
calorie
"type": "upKcal",
"date": "120", [str] // Calories (kcal)

blood pressure
"type": "upBP",
"date": "120/80/88", [str] // Blood pressure (systolic/dynamic/pulse or systolic/dynamic)

blood oxygen
"type": "upBO",
"date": "96", [str] // Blood oxygen

temperature
"type": "upBodyTemperature",
"date": "36.8/31.6/28.2", [str] // Body temperature
(body surface temperature/environmental temperature, with one decimal place retained)

blood sugar
"type": "upBS",
"date": "9.6", [str] // Blood sugar

blood fat
"type": "upBF",
"date": "3.12/1.20/1.10/3.12", [str] // Blood lipid (total cholesterol concentration, high density lipoprotein cholesterol concentration

```
 purine trione
"type": "upUA",
Data: "441", [str] // uric acid
```

UpBatch

Explanation: The device uploads health data in batches

```
{
    key
    "type":"upBatch",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "testType" *: 1, [int][*] // Measurement type (0: timed measurement 1: hardware manual measurement 2: app command
    "WearPosition" *: 1 [int][*] //Wear position of the watch (1=left hand, 0=right hand) (optional parameter
    "airbagSize" *: 1 [int][*] // airbag size (1= small, 2= medium, 3= large) (optional parameter)
    "Timestamp": 1648111390074 // Data upload timestamp}

Key: Different health data types Key are the following
fields heart rate
"dataType": "upHeartRate",
"data": "100,98,97", [str] // heart rate values (up to 500 sets)
"dataTime": "1648111390075,1648111390073,1648111390074", [str]// Data measurement time

// The above three fields are extended fields. If the device needs to upload a batch of heart rate data, the complete transmission data is as follows:
{
    "type":"upBatch",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "testType" *: 1, [int][*] // Measurement type (0: timed measurement 1: hardware manual measurement 2: app command
    "WearPosition" *: 1 [int][*] //Wear position of the watch (1=left hand, 0=right hand) (optional parameter
    "airbagSize" *: 1 [int][*] // airbag size (1= small, 2= medium, 3= large) (optional parameter)
    "dataType": "upHeartRate",
    "data": "100,98,97", [str] // heart rate values (up to 500 sets)
    "dataTime": "1648111390075,1648111390073,1648111390074", [str]// Data measurement
    "timestamp": 1648111390074 // Data upload timestamp
}
```

Up sleep data

Explanation：

Upload sleep data according to the status of the sleep setting in "Device Configuration". (Reports can be made based on the preset sleep end time "sleepEndTime". If there is no sleepEndTime, it is recommended to report at a random time between 10:00 and 11:00 AM.)

Or report the current situation when appropriate. If you are asleep, the reporting time will be delayed for 15 minutes before judgment, and the report will be made when the sleep state is awake.

```
{
    "type": "upSleep",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "IsAccumulative": 1, [*][int] //Optional field. If not specified, default accumulation is applied. This field indicates whether the system should accumulate data based on the sleep cycle.
    "startTime": 1653228000000, [long] //The first recorded sleep start time (in milliseconds).
    "endTime": 1653271200000, [long] // Last recorded end time of sleep (mandatory)
   "dateTime": [Array] // sleep segment data array
      {
        "startTime": 1653228000000, [long] // Start time of this sleep type (millisecond timestamp)
            "end time": 1653271200000, [long] // The end time of this sleep type
        "duration": 30, [int] // How many minutes it lasts
        "sleeptype": "deepSleep" [str] // sleep type},
        {
        "startTime": 1653228000000, [long] // Start time of this sleep type (millisecond timestamp)
            "end time": 1653271200000, [long] // The end time of this sleep type
        "duration": 60,
        "sleepType": "lightSleep"
      }
    ],
    "timestamp": 1648111390074
}

Sleep type sleepType description
Deep sleep: sleepType = "deepSleep"
Light sleep: sleepType = "lightSleep"
REM: sleepType = "rem" and
sober: sleepType = "sober"
```

Upload ECG analysis data

Explanation：

The equipment actively reports the ECG analysis data

```
{
    "type":"upECGAnalysis",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "devType":"xxx",
    "mealstatus": "-1", // indicates the current meal status: -1 (not filled), 1 (fasting in the morning),
    2 (pre-meal), 8 (with medication). "medicationstatus": "-1", // indicates the current medication status: -1 (not filled), 0 (no medication taken today), 1 (medication taken today).
    "fileBase64": "" (ECG data)
```

```
    "fileName": "N5AB2454.lp4", // file name
    "dataStatus": 0, [int] // This packet data status 0: first packet data 1: intermediate packet data 2: end packet data
    (" timestamp: 1648111390074
}
```

upTodayActivity

Explanation：

Periodic reports are made according to the step frequency set in "Set device measurement frequency"

```
{
    "type": "upTodayActivity",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "step": 5000, [int] // steps
    "exerciseTime": 3600, [*] [int] // Exercise time (unit: seconds)
    "standTime": 10, [*] [int] // Standing duration (unit: minutes)
    "timestamp": 1648111390074
}
```

upRun data

Explanation：

The motion data is opened and recorded by the equipment actively, and the report is automatically submitted after the recording.

```
{
    "type": "upRun",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "startTime": 1736317764000, // start time, milliseconds.
    "endTime": 1736317764000, // end time, milliseconds
    "exerciseTime": 3600, [int] // Exercise time in seconds
    "consumed": 245, [int] // consumed (unit: kcal)
    "mileage": "2.8", [string][*] //mileage (unit: km, with one decimal place) "speed":
    {[*] //speed
        "maxPace": 5.5, [double] // Maximum pace (unit: minutes/km, rounded to one decimal place)
        "avgSpeed": 3.25, [double] // Average speed (unit: km/h, two decimal places retained)
        "avgPace": 10.5 [double] // Average pace (unit: min/km, one decimal place retained)
    },
    "Steps": {[*] // steps
        "stepNumber": 5180, [int] // steps (unit: steps)
        "avgStrideFrequency": 74, [int] // Average stride frequency (unit:
        steps/min)
        "avgStride": 71, [int] // Average stride length (unit: cm)
```

```
        },
        "heartRate": {// Heart rate
            "time": [5,10], [array] // time array (e.g., 5 for the fifth minute, 10 for the tenth minute)
            "vale": [100,106], [array] // heart rate array (e.g., 100 for the fifth minute's heart rate)
            "limit": 3, [int] // limit (unit: minutes)
            "anaerobic": 8, [int] // anaerobic endurance (minutes)
            "aerobic": 21, [int] // aerobic endurance (minutes)
            "fatBurn": 14, [int] // Fat burning (minutes)
            "warmUp": 12 [int] // Warm-up (minutes)
        }
        "timestamp": 1648111390074
}
```

explain：

Running: type = upRun
Walking: type = upWalk
Cycling: type = upRide
Free movement: type = upFree
rope skipping: type = upRope
Badminton: type = upBadminton
Table tennis: type = upTable
Tennis Tennis: type = upTennis
Climbing: type = upClimb
Basketball: type = upBasketball
Volleyball: type = upVolleyball
Dancing: type = upDance
Dynamic bike: type = upSpinningBike
Yoga: type = upYoga
Kicking: type = upJumpingJack
Sit-ups: type = upSitUps
Football :upFootball
Wushu: upWushu
Taekwondo machine:
upATaekwondo
Machine
Taiji jumping:upTaijiquanJumping
HulaHoop
:upHulaHoop

Running and walking require mileage, speed and step to be uploaded

UpLocation

Explanation：

1. The device reports data according to the set "location interval time"

2. The device performs a positioning report according to the instruction for positioning received from the server

3. GPS, WIFI, and baseStation: Among the three, GPS and baseStation must be filled in one, while WIFI is optional. The information required for WIFI positioning (WIFI data must have at least 5 or more) should contain at least one baseStation information. When wifi parsing fails, the base station location will be parsed.

The order of parsing is GPS> WIFI> baseStation

```
{
    "type": "upLocation",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "baseStationType": 0, [int] // Is CDMA (0: no, 1: yes)
    "position DataType": "1", [str] // Position data type: 0: periodic trajectory report 1: instruction query response
    "Wifi": [] // A set of WiFi location data
        {
            "ssid": "sb", [str] // The name of the wifi        "ssid": "sb", [str] // The name of the wifi
            "signal": "-75", [int] // WiFi signal strength     "signal": "-75", [int] // WiFi signal strength
            "mac": "f8:8c:21:ef:fd:83" [str] // The MAC address of the wifi
        },
        {
            "ssid": "sb2",
            "signal": "-52",
            "mac": "c8:3a:35:ba:f1:23"
        }
    ],
    "baseStation": [[array][*] // Base station location data set
        {
            "mcc": 460, [int][*] // This is only available in non-CDMA cases and is a national number
            "mnc": 0, [int][*] // The operator number is only available in non-CDMA cases
            "lac": 9475, [int][*] // Only available in non-CDMA scenarios; cell ID
            "ci": 61009, [int][*] // Only available in non-CDMA scenarios; base station ID
            "rxlev": 51, [int][*] // Only available in non-CDMA scenarios; signal strength
            "sid": 13832, [int][*] // Only available in CDMA scenarios; system identification
            code
            "bid": 688, [int][*] // Only available in CDMA scenarios; network identification
            code
            "nid": 65535 [int][*] //CDMA only has this, the unique identification code of the cell
        }
    ],
    "gps": {[*] //GPS positioning data
        "lon": "113.4201386", [str] // longitude DDD.DDDDDD format;
        "lat": "23.1782397", [str] // latitude DD.DDDDDD format
        "height": 100, [int] // altitude in meters
        "satelliteNum": 5, [int] // indicates the number of GPS satellites
        "GSM": 100 [int] // GSM signal strength
        "Type": 0, [int] // Coordinate system: 0 Global GPS 1 Gaode 2 Baidu 3 Google 4 Tencent
    },
    "timestamp": 1648111390074
}
```

UpBattery (upload at regular intervals)

```
{
    "type": "upBattery",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "batteryLevel": 100, [int] // Power
```

```
    "batteryState": 1, [int] // Charging status: 0 (not charged) 1, Charging. "batteryType": 2, [int] // Data type: 0 (on); 1, Off; 2. Report on time; 3, Low power
    "timestamp": 1648111390074,

}
```

Get weather-upWeather

Explanation：

If the device is not located at present, the data cannot be obtained

```
{
    "type": "upWeather",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "timestamp": 1648111390074
}
```

Power off the system-upShutdown

Explanation：

When the device shuts down by itself, or when it automatically shuts down due to low power, it needs to be reported once.

```
{
    "type": "upShutdown",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "timestamp": 1648111390074
}
```

SMS report-upSMS

Explanation：

The msgContent field uses a directly readable string and is not encoded. E.g., "SMS content XXXX"

```
{
    "type": "upSMS",
        "imei": "865028000000306"

                                    ,
```

```
    "deviceModel": "xxx",
    "sender: "xxxxxxxxxxxxxxxxxx", [str] // Check the results of call charge and data traffic
    "timestamp": 1648111390074


}
```

The device actively resets to factory Settings - upReset

Explanation：

The device restores itself to factory Settings and executes the restore to factory Settings instruction after receiving the server response.

```
{
    "type": "upReset",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "status": 0, // 0: Clear device data 1: Clear device data and unbind the relationship
    with the user Timestamp": 1648111390074
}
```

Remote pickup report (not connected yet) - upPickup

```
{
    "type":"upPickup",
    "imei":"865028000000306",
    "deviceModel": "xxx",
    "bytes": [], [byte] //Remote pickup and upload audio byte
    data "Timestamp": 1648111390074
}
```

Check out the latest sleep command- upSleepFind

For example, the device uploads electrocardiogram data or sensor commands to query the latest sleep of the day

```
{
    "type":"upSleepFind",
    "imei":"865028000000306",
    "deviceModel": "xxx",
    "timestamp":1648111390074
}
```

Check out the latest OTA configuration-upGetOTA

If the device supports OTA upgrade function, the OTA configuration should be obtained at startup. After successful acquisition, the command "OTA Configuration issued" will be issued

```
{
    "type":"upGetOTA",
    "imei":"865028000000306",
    "deviceModel": "xxx",
    "timestamp":1648111390074
}
```

Request to send the default device configuration-upGetDevConfig

This command should not be sent frequently. You can send the request to synchronize the configuration during device initialization or when the device is turned on.

If the user has set the configuration, the latest configuration is delivered. If not, the default configuration for this type of device is obtained in the background

After the request is successful, it will be sent as follows:

Location report interval, device measurement frequency, and device configuration

```
{
    "type":"upGetDevConfig",
    "imei":"865028000000306",
    "deviceModel": "xxx",
    "timestamp":1648111390074
}
```

Device sends sensor value command-upSensorValue

```
{
    "type":"upSensorValue",
    "imei":"865028000000306",
    "sensorType": int, //[int] Sensor type: e.g. 0: triaxial gyroscope 1: triaxial accelerometer (e.g.,
    "date": "x, y, z", [str] corresponding sensor values, X, Y, Z are the angular velocities in three directions
    of the triaxial gyroscope (e.g. "timestamp": 1648111390074
}
```

Device batch upload sensor values command-upSensorValues

```
{
    "type":"upSensorValues",
    "imei":"865028000000306",
    "sensorType": "0,1", [str] Sensor type: (e.g., 0: triaxial gyroscope, 1: triaxial accelerometer). Corresponding sensor values are "data": "x, y, z; x, y, z" [str], where X, Y, Z represent the angular velocity in three axes of the triaxial gyroscope.
    "dataTime": "1648111390073,1648111390075" [str], indicating the timestamp of data measurement
    "timestamp":"1648111390074"
}
```

Call records are reported - upCallLog

```
{
    "type":"upCallLog",
    "imei":"865028000000306",
    "name": "", // Number of the owner (if unknown, enter the number)
    "phone": "", // Target phone number
    "beginTime": 1648111390074, // call start time stamp
    "endTime": 1648111390074, // call end time stamp
    "duration": 60, // call duration (seconds), default 0
    "waitDuration": 0, // Waiting duration (seconds), default 0
    "callType": 0, // Call type (0-incoming, 1-outgoing)
    "isSwitchOn": 0, // Is connected (0-not connected, 1-connected)
    "timestamp": 1648111390074
}
```

The device reports configuration-upDeviceConfig

Function items that can be set on the synchronization device can be set to the server

```
{
    "type": "upDeviceConfig",
    "imei": "",
    "configs": {},
    "timestamp":1648111390074
}
// The following is the expanded value in the configs field
Sleep Settings
"SleepIntervalOrSwitch":{
    "switchState": 1, [int] Switch status: 0 off and 1 on
    "sleepStartTime": "220000", // [str][*] Sleep start time (format HHmmss here represents the previous
    "sleepEndTime": "100000" // [str][*] Sleep end time (format HHmmss here represents today
```

```
}
 For example：
// If the device uploads the sleep setting, the complete object is as follows
{
    "type": "upDeviceConfig",
    "imei": "",
    "configs": {
        "SleepIntervalOrSwitch":{
            "switchState": 1, [int] Switch status: 0 off and 1 on
            "sleepStartTime": "220000", // [str][*] Sleep start time (format HHmmss
            "sleepEndTime": "100000" // [str][*] Sleep end time (format HHmmss
        },
    },
    "timestamp":1648111390074
}
```

The device actively obtains the device binding status-upGetDevBindStatus

The server will issue the dnDevBindStatus command

```
{
    "type": "upGetDevBindStatus",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    "timestamp": 1648111390074
}
```

App application package name report-upAppIdent

```
{
    "type": "upAppIdent",
    "imei": "865028000000306",
    "appList":[
        {
            "appName": "",// application name
            "package name": "",// application package name
            "status": 1, // whether to enable status 1-yes, 0-no}
    ],
    "timestamp": 1648111390074
}
```

Click the "friend" instruction to report-upMakeFriend

Explanation: After successful friendship, the "downSyncFriend" synchronization instruction will be sent to synchronize the friend list

When using "TouTou Friends" within a certain interval (currently set to 30 seconds by default), the system will not respond to the "TouTou Friends Result: Send Down MakeFriend" command when a single watch is uploaded. The system will only respond with "TouTou Friends Result: Send Down MakeFriend" when paired watches are uploaded.

The "downMakeFriend" command

```
{

    "type"    : "upMakeFriend",
    "imei": "xxx",
    "timestamp": 1728615101017, // Current unix timestamp
    "baseStationType": 0, [int] // Is CDMA (0: no, 1: yes)

    "gps": <object>[*] // Location information is the same as upLocation, and upLocation is the standard

    "baseStation": <array>[*] // Same as upLocation's location information, with upLocation as the standard.

    "wifi": <array>[*] // Same as upLocation's location information, with upLocation as the standard

}
```

Click the friend synchronization command-upSyncFriend

Explanation: The server will send the "touch to make friends synchronization instruction downSyncFriend" to synchronize the friend list, and the watch can actively obtain the latest data when entering the friend list

```
{
    "type"  : "upSyncFriend",
    "imei": "860748051983354",
    "timestamp": 1728627057880
}
```

Click the delete instruction to delete a friend-upDelFriend

Explanation:: The server will send "touch friends delete result down-delfriend" to synchronize the friend list

```
{

    "type"  : "upDelFriend",
    "imei": "860748051983354",
    "timestamp": 1728627057880,
    "targetimeiarr": [<array> // The set of IMEI numbers to be removed
```

```
        "1234567890000001",
        "1234567890000002",
    ...
  ]
}
```

Star Calendar-UpEphemeris

```
    {
        "imei": "864688111111111",
        "type": "upEphemeris",
        "date": "111", // Astronomical values
        "timestamp": 1751351140365
    },
```

The server replies to instruction

Custom extension instruction (issued) -dnCustom

```
{
    "type": "dnCustom",
    "imei": "865028000000306",
    "deviceModel": "xxx",
    ... Custom parameters
    "timestamp": 1648111390074
}
```

Issue the watch status of the device-dnDevBindStatus

Explanation:

When the watch is bound, a binding status notification is sent (the device will also receive the binding status in the login instruction reply when logging in).

```
{
    "type":"dnDevBindStatus",
    "imei":"865028000000306",

      "status": 0, // status: 0: not bound 1: bound
    "timestamp": 1648111390074

}
```

Issue the ECG analysis results- dnECGAnalysis

Explanation：

When the equipment reports that the ECG analysis data has been analyzed, the analysis results will be sent out

```
{
    "type":"dnECGAnalysis",
    "imei":"865028000000306",
    "heartRate": "100", // heart rate bpm
    BSc: "9.6", // blood glucose value mg/dl (mmal/L=mg/dL/18)
    "BScIndex": "3", // blood sugar: 1 high, 2 low, 3 pay attention, 4 good
    "BPcDia": "120", // systolic value mmHg
    "BPcSys": "80", // diastolic value mmHg
    "timestamp": 1648111390074
}
```

Request to measure health data-dnHeartRate

Explanation：

Use it in conjunction with "Upload health data" and pay attention to the distinction between type types

```
{
    "type":"dnHeartRate",
    "imei":"865028000000306",
    "timestamp": 1648111390074
}

type explain ：
Heart rate: type = "dnHeartRate"
Blood pressure: type = "dnBP"
Blood oxygen: type = "dnBO"
Body temperature: type = "dnTemperature"
Breathing rate type = "dnBreathe" // After batch uploading heart rate, the breathing rate is measured and sent out
================================
// The following four types have specific optional parameters
ECG electrocardiogram: type = "dnECG"
```

```
hrv heart rate variability: type = "dnHRV"
PPG photoelectric capacitive volume pulse wave recording: type = "dnPPG"
RRI ventricular beat interval: type = "dnRR"

"frequency": "200", [str] // Sampling frequency Hz, if not used, the default frequency of the device
"oneTime" *: 300, [int] //single sampling time in seconds. If not used, the default
sampling time of the device is "collectionLogo" *: "87654321", [str] //the identifier
of this sampling
```

Request location-dnLocation

```
{
    "type":"dnLocation",
    "imei":"865028000000306",
    "timestamp": 1648111390074
}
```

Set the location interval time

Explanation：

Refers to the instruction that sets the interval between devices to upload the positioning data once after a certain period of time

```
{
    "type":"locationInterval",
    "imei":"865028000000306",
    "intervalTime": 1440, [int] // Positioning interval in seconds. When it is 0, it is turned
    off. timestamp: 1648111390074
}
```

Set the device measuring frequency

Explanation：

Note that different types are distinguished. After setting, the device uses the corresponding "device upload command" to report according to the measurement frequency and type

```
{
    "type": "deviceMeasuringFrequency",
    "imei": "352273017386001",
    "configs":{
        "upHeartRate":{
            "interval": "60" [str] // Interval duration (unit: minutes)
```

```
            }
        },
        "timestamp": 1648111390074
}
```

Configs Explain:

Heart rate

```
"upHeartRate":{
        "interval": "60" [str] // Interval duration (unit: minutes)
}

//ecg
//"upECG":{
// "Interval:" 60" [str] // Interval duration (unit: minutes)
//}
//hrv
//"upHRV":{
// "Interval:" 60" [str] // Interval duration (unit: minutes)
//}
//ppg
//"upPPG":{
// "Interval:" 60" [str] // Interval duration (unit: minutes)
//}
//rr
//"upRR":{
// "Interval:" 60" [str] // Interval duration (unit: minutes)
//}
```

Blood pressure

```
"upBP":{
        "interval": "60" [str] // Interval duration (unit: minutes)
}
```

Temperature

```
"upBodyTemperature":{
        "frequency": 5, [int][*] // The collection frequency. After collecting the corresponding number of times, the median is taken
        "interval": "60" [str] // Interval duration (unit: minutes)
}
```

Blood Oxygen

```
"upBO":{
        "frequency": 5, [int][*] // The collection frequency. After collecting the corresponding number of times, the median is taken
        "interval": "60" [str] // Interval duration (unit: minutes)
}
```

Steps

```
"upStep":{
        "interval": "60" [str] // Interval duration (unit: minutes)
}
```

Rate of Breathe

```
"upBreathe":{
        "interval": "60" [str] // Interval duration (unit: minutes)
}
```

Set the device configuration-deviceConfig

```
{
    "type": "deviceConfig",
    "imei": "352273017386001",
    "configs":{
        "StepTarget":{
            "steps": 6000
        }
    },
    "timestamp": 1648111390074
}
```

Configs Explain:

Steps target

```
"StepTarget":{
    "steps": "6000" [str] // Target number of steps
}
```

Exercise time goals

```
"ExerciseTimeTarget":{
    "duration": "30" [str] // Target time (unit: minutes)
}
```

Contextual model

```
"SceneMode":{
    "sceneMode": 3 [int] // Number: 1: vibration 2: ringtone 3: ringtone + vibration 4: no vibration
    and no sound
}
```

Trajectory switch (not available)

```
"TrackSwitch":{
    "switchState": 0 [int] //Switch status: 0: off, 1: on
}
```

Low power alarm threshold setting

```
"LowPower": {
    "Battery": 60 [int] // Threshold: Trigger a low battery alarm message when the power is below the
    threshold%
}
```

Falling detection switch

```
"FallWarnSwitch":{
    "switchState": 0 [int] //Switch status: 0: off, 1: on
}
```

Refuse to answer the stranger switch

```
"CallInLimitSwitch":{
    "switchState": 0 [int] //Switch status: 0: off 1: on}
```

Voice reminder reply switch (not available)

```
"RemindReplySwitch": {
    "switchState": 0 [int] //Switch state: 0 off and 1 on
}
```

SOS SMS switch

```
"SOSSwitch":{
    "switchState": 1 [int][*] // SMS switch
}
```

```
SOS rotation count
"SOSRotation":{
        "rotationNums": "3" [str][*] // Number of rotations
}
SMS upload switch (controls whether to upload SMS messages for inquiry
about phone charges)

"MessageSwitch":
        "switchState": 1 // Switch state: 0 off and 1 on
  }
Call answering mode
"CallAnswerMode":{
        "callAnswerMode": 1 [int] //call answer mode 0 disable automatic answer,
1 answer immediately, 230 seconds
}
Call-in limit switch
"CallInLimitSwitch":{
        "switchState": 1 //Switch state: 0 Off 1 On After opening, only family number and
SOS number can be called in
}
Sleep Settings
"SleepIntervalOrSwitch":{
        "switchState": 1, [int] //Switch state: 0 is off and 1 is on
        "sleepStartTime": "220000", [str][*] //Sleep start time (format HHmmss, here representing
        the previous
        "sleepEndTime": "100000", [str][*] //Sleep end time (format HHmmss, here representing today
        "sleepTarget": 480 [int] //Sleep target duration (minutes), default 8 hours
}
Continuous blood oxygen monitoring at rest
"ContinuousBOCheck": {
        "switchState": 0, [int] //Switch status: 0 off and 1 on
}
Heart rate was measured continuously
"ContinuousHRSwitch": {
        "switchState": 0, [int] //Switch status: 0 off and 1 on
}
Blood pressure measurement reminder
"BPTimerMeasure": {
        "switchState": 1, [int] //1=on 0=off
        "amRemindTime": "07:00",          [str][*] // Morning measurement reminder time (format
                                          mm:ss)
        "pmRemindTime": "18:00",          [str][*] // Afternoon measurement reminder time (format
                                          mm:ss)
}
Blood pressure warning
"BPEarlyWarning": {
        "switchState": 1,  [int] "hpWarn":
        135,  [int][*]                    //1= On 0= Off//
                                           High pressure
                                           warning value
        "LPWarn": 90, [int][*] // Low pressure warning value
}
Heart rate warning
"HROvertopRemind": {
        "switchState": 1, [int][*] //Heart rate high reminder (1=on 0=off)
        "remindValue": 120, [int][*] //Reminder value
        "exerciseSwitchState": 1 [int][*] // Exercise heart rate is too high reminder (1=Smart reminder 0=Off)
        "exercise HRMin": 100, [int][*] // minimum exercise heart rate
        "exercise HRMax": 140, [int][*] // maximum exercise heart rate
          "exerciseRemindValue": 140, [int][*] // Exercise heart rate high alert value
}
 Dial
"dialSwitch":{

        "switchState": 1 // Switch status: 0 off and 1 on
```

```
  }
PPG blood pressure trend
"PPGBPTrend": {
        "switchState": 1, //1= on and 0= off
}
Blood management health risk warning
"CardiovascularWarn": {
        "switchState": 1, //1= on and 0= off
},
Set the value for blood oxygen anomaly alert
"bloodOxygenWarn":{
          "switchState": 1, //1= on and 0= off
          "reminderValue": 90, [int][*] // Reminder value (minutes)}
}

Temperature automatic monitoring switch
"ContinuousTempSwitch":{
        "switchState":0, [int] //Switch state: 0 off and 1 on
}
A high temperature is a reminder
"TemperatureExceedRemind":{
        "switchState": 1, [int] // High temperature reminder (1 on 0 off)
          "RemindValue": 38.5, [double][*] // The reminder value of high body temperature
}
Low body temperature alert
"TemperatureBelowRemind":{
        "switchState": 1, [int] // Low body temperature reminder (1=on 0=off)
          "RemindValue": 38.5, [double][*] // Low body temperature reminder value
}
Low heart rate alert
"HeartRateBelowRemind": {
        "switchState": 1, [int][*] // Low heart rate reminder (1=on, 0=off)
        "remindValue": 120, [int][*] // Reminder value
        "exerciseSwitchState": 1, [int][*] // Exercise heart rate too low reminder (1=on 0=off)
        "exercise HRMin": 100, [int][*] // minimum exercise heart rate
        "exercise HRMax": 140, [int][*] // maximum exercise heart rate
        "exerciseRemindValue": 140, [int][*] // Exercise heart rate low reminder value
}
Remind of regular heart rate measurement
"ECGTimerMeasure": {
        "switchState": 1, [int] //1=on 0=off
        "remindTime": "07:00", [str]// start time (format HH:mm) Time range: 7:00~22:00}
}

Unlock the password
"UnlockPassword":{
        "switchState": 1 // Whether to turn off the device lock screen. 0-No,1-Yes
  }
  Volume Settings
  "volumeSetting":{
          "value":51
  }
  Automatic wrist switch
  "wristLiftSwitch":{
          "switchState": 1 // (1= on, 0= off)
  }
  12/24 time system
```

```
 "timeFormatMode":{
        "value":24 //24/12
      }
findMyFunction
"findMyFunction": {"switchState": 1} // (1= on, 0= off)

Power saving mode
"powerSaveMode": {"switchState": 1} // (1= on, 0= off)

 Device language
"deviceLanguage":{"language": "cn"} //cn Chinese, en English Device lost

"watchLost": {"switchState": 0} // (1= on, 0= off)
```

Set an alarm clock

Explanation：

The number of alarm clocks currently supports up to 10 groups (tentative)

```
{
      "type":"alarmClock",
      "imei":"352273017386001",
      "alarmClockList": [[array] //Alarm clock array
            {
                  "label": "It's time to eat", [str] // Alarm clock name
                  "startTime": "08:00", [str] // Start time in 24 hours, in the format of "hh: mm"
                  "week": "1111100", [str]// Number of repeated weeks
                  "status": "1", [str]// Status (1: On, 0: Off)
                  "url": "http://xxx.mp3", [str][*] // Voice reminder file address
            }
      ]
      "timestamp": 1648111390074
}

Explain：
Week: A 7-character string consisting of 0 and 1, where 0 means off and 1 means on.
For example, "1111100" means that the week is open from Monday to Friday
```

Set family Number

Explanation：

Similar to the whitelist/ address book, currently up to 10 groups (tentative) are supported

```
{
      "type": "familyNumber",
        "imei": "352273017386001"

                                    ,
```

```
        "familyNumbers": [[array] // Family number array
                {
          "familyNumberId": "8c67b51b", [str] //family number id "name": "son", [str]
    //contact name
          "phone": "12345678910", [str]// contact phone number
          "sosSwitch": 0, [str][*] // Whether to set as SOS number "
          areaCode": "86", [str]// International area code, such as China:86
                }
        ],
        "timestamp": 1648111390074




}
```

Set the SOS number

Explanation：

Make a choice from the family number.

```
{
        "type": "SOSNumber",
        "imei": "352273017386001",
        "sos Numbers": [[array] //SOS number set
                {
                        "sos Number Id": "8c67b51b", [str] //sos Number ID
                         "name": "son", [str] //contact name
                        "phone": "12345678910", [str]// contact phone number
                }
        ]
        "timestamp": 1648111390074
}
```

Check your phone bill/traffic-find PhoneBill Or Flow

Explanation：

The device sends a text message to the corresponding operator according to the type of card inserted for query. If it is an Internet of Things card, ignore it

```
{
        "type": "findPhoneBillOrFlow",
        "imei": "352273017386001",
        "queryType": 1, [int] // query type: 0: check the phone bill 1: check the traffic
        "timestamp": 1648111390074
}
```

Send sleep data-dnUpSleep

```
{
"type":"dnUpSleep",
"imei": "352273017386001",
"upDayStr": "2023-10-31", // Which day the current sleep data belongs.
"value": "200/48/152/0", // Sleep: total, deep sleep, light sleep, awake
"timestamp": 1648111390074
}
```

Issue weather data-dnWeather

```
{
        "type": "dnWeather",
        "imei":"352273017386001",
        "iIsCDMA": "0", [str][*] // Is it CDMA? (optional)
        "weather": "Cloudy", [str] // Weather phenomenon (Chinese
        description)
        "weatherType": 1, [int]// Weather type 0: Sunny 1: Cloudy 2: Wind 3: Rain 4: Snow 5: Cloud 6: Fog 7:
        "province": "Beijing", [str] // Province "city":
        "Dongcheng District", [str] // City
        "adcode": "110101", [str] // Regional code
        "temperature": "26", [str] // real-time temperature, unit: degrees Celsius
        "wind direction": "southwest", [str] // Wind direction, winddirection code corresponding description
        "windpower": "5", [str] // Wind power, which returns the windpower code corresponding to the wind power level
        "humidity": "56", [str] // Air humidity
        "daytemp": "32", [str] //Maximum temperature (highest daytime
        temperature)
        "nighttemp": "25", [str] //Minimum temperature (lowest
        nighttime temperature)
        "reporttime": "2018-09-01 18:00:00" [str] // Data release time
        "timestamp": 1648111390074
}
```

Weather type comparison:

https://lbs.amap.com/api/webservice/guide/tools/weather-code/

Find the device

```
{
        "type": "find",
        "imei":"352273017386001",
        "timestamp": 1648111390074
}
```

Restore factory Settings-reset

Explanation：

After receiving the restore factory Settings instruction, the device should first reply to the service, and then disconnect the connection to perform the restore factory Settings operation

```
{
    "type": "reset",
    "imei":"352273017386001",
    "timestamp": 1648111390074
}
```

Restart

Explanation:：

After receiving the restart instruction, the device should first reply to the service, and then disconnect the connection to perform the restart operation

```
{
    "type": "restart",
    "imei":"352273017386001",
    "timestamp": 1648111390074
}
```

Remote shutdown-powerOff

Explanation：

After receiving the remote shutdown command, the device should first reply to the service, and at the same time need to upload a shutdown location report, and then disconnect the connection to execute the remote shutdown operation

```
{
    "type": "powerOff",
    "imei":"352273017386001",
    "timestamp": 1648111390074
}
```

Message notification-msgNotice

Explanation：

Messages that can be displayed on the device shall be discussed in detail by the UI specific device scheme

```
{
    "type": "msgNotice",
    "imei":"352273017386001",
    "msgType": "msg" [str] //Message type "msg":
    "xxxxx", [str] //Message content "Timestamp":
    1648111390074
}

 explain：

Ordinary news
msgType = "msg"
Blood pressure
measurement
msgType = "BP",
Blood oxygen
measurement
msgType = "BO",
Motion mode
msgType = "run"
Electrocardiogram
measurement
msgType = "ECG"
```

OTA configuration is deployed-OTA

Explanation：

This OTA configuration is configured by the APP. After issuing instructions, the device can decide how to update the package according to the configuration reference.

This instruction is also issued when the APP saves and modifies the remote upgrade configuration

```
{
"type": "OTA",
"imei":"352273017386001",
"autoUpdate": 1, [int] Whether to update automatically
"useWIFI": 1, [int] Whether to useWIFI
"useData Flow": 1, [int] Whether to use data flow to download
"url": "xxx", [str] New version download address
"md5sum": "XXXXXX", [str] md5 checksum
"fileSize": 1345, [int] file size (bytes)
"versions": "2.1.7", [str] version number of the new version
"content": "xxx", [str][*] content description of the new version
"timestamp": 1648111390074
}
```

Issue the breathing rate-dnUpBreathe

```
{
    "type":"dnUpBreathe",
    "imei": "xxxx",
    "value": "20", // respiratory rate
    "timestamp": 1648111390074
}
```

Issue the medication plan-dnMedicationPlan

```
{
    "type":"dnMedicationPlan",
    "imei": "xxxx",
    "drugType": 0, [int] Drug type. 0: hypertension medication, 1: diabetes medication, 2:
    hyperlipidemia medication, 3: hyperuricemia medication.
    "drugName": "", [Stirng] Drug name
    "drugDose": 0.0, [Double] Drug dose (optional)
    "drugUnit": "0", [String] Drug unit (optional) 0: particle, 1: branch, 2: ml, 3: mg, 4: IU, 5: other
    "drug StartTime": "2024-02-23", [String] medication start time
    "drugEndTime": "2024-02-23", [String] medication end time
    "drugInterval": 0.0, [Double] medication interval
    "drugTime": // [json] period
            "alarmClock": {
                "Midday": "14:39", // noon
                "Night": "14:39", // evening
                "Before sleep": "14:39", before
                "Morning": "14:39" // morning
        },
        "checkboxes": [3], [int] Selected value, multiple selection 0: morning, 1: noon, 2: evening,
        3: evening "radio": 0 [int] 0: before meal, 1: after meal
                },
    "timestamp": 1648111390074
}
```

Class is disabled (1) -disturb- (used in the old version, please use disturb_two for the latest access)

Explanation:

Turn on Do Not Disturb. During the disabled period, the device will reject all calls and messages, but you can set up SOS numbers to call in and out.

```
{
    "type":"disturb",
    "imei":"352273017386001"
    "disturb": {                                ,


        "sosIsCallOut": 1, [int] Whether SOS can be called out during the disable period? (0-Disabled, 1-Enabled)
        "sosIsCallIn": 1, [int] Whether SOS can be called in during the disable period? (0-Disabled, 1-Enabled)
        "timeSlot": [// Array] Time slot array
            {
              "week": "0011000", // Number of repeated weeks,
              "startTime": "18:18", // Start time, 24-hour format, in the format of "hh: mm"
              "endTime": "18:18", // End time, 24-hour format, in the format of "hh: mm"
            },
            ...
        ],
        "status": 1 [int] Do not disturb total status (0-off, 1-on)
    }
    "timestamp": 1648111390074
}

explain：
week: A 7-character string consisting of 0 and 1, where 0 means off and 1 means on.
For example, "0111110" means that the week is open from Monday to Friday
```

Class disable (2) -disturb- two

Explanation：

Turn on Do Not Disturb, and the device will reject all calls and messages during the disable period.

```
{
    "type":"disturb_two",
    "imei":"352273017386001"            ,
    "disturb": {


              "timeSlot"[ //[Array] An array of time periods
              "week": "0011000", // Number of repeated weeks,
              "startTime": "18:18", // Start time, 24-hour format, in the format of "hh: mm"
              "endTime": "18:18", // End time, 24-hour format, in the format of "hh: mm"
            },
            ...
        ],
        "status": 1 [int] Do not disturb total status (0-off, 1-on)
    }
    "timestamp": 1648111390074
}

Explain：
week: A 7-character string consisting of 0s and 1s, where 0 means off and 1 means on.
For example, "0111110" means that the week is on from Monday to Friday
```

Remote photography

```
{
        "type": "photography",
        "imei":"352273017386001",
        "mute": 0, // whether to mute the photo, this value exists is 0 and
        for silent photo
        "timestamp": 1648111390074
}
```

AutoAnswer

```
{
        "type": "autoAnswer",
        "imei":"352273017386001",
        "switchState": 0, //switch state: 0 off and 1 on
        "timeoutPeriod": 10, // The timeout time (seconds). If someone calls the device and the device does not
        answer for 10 seconds, the device will
        "timestamp": 1648111390074
}
```

DownAppControl

The device will send first reports for"App application package name - upAppIdent" command

```
{
    "type": "downAppControl",
   "imei":"352273017386001",
    "appList":[
            {
   "app": "",//<string>, application identifier, reference upAppIdent instruction
   "status": <integer> //show (0: no, 1: yes)
            }
       ]
        "timestamp": 1648111390074
}
```

Set Flower

```
{
    "type": "setFlower",
```

```
    "imei":"352273017386001",
    "redflower": 2, // Number of red flowers
    "timestamp": 1648111390074

}
```

Click on the results to send down MakeFriend

Explanation: If the friend request fails, it will not be sent out. Otherwise, when there are many people, there will be simultaneous friend requests at A point and A1 point, B point and B1 point. Then there will be A1 and B, A1 and B1, followed by failed sending out, and then A1 and A ==> successful sending out

```
{

    "type"  : "downMakeFriend",

    "success": 1, //1 success or-1 repeated friend request (failure will not be sent)

    "successimei": <str>[*] // Only available when a friend request is successful

    "failimei": <str>[*] // Only a field that is repeated when making friends

    "timestamp":1728614143505 }
```

Bumper Dating Synchronous Command Issued - downSyncFriend

```
{

    "type"  : "downSyncFriend",
    "imei":"860748051983354",
    "timestamp":1728614142032,
    "sync friends": <array> [// List of all friends

        {

            "imei": "1234567890000002",  <str>//imei

            "time": The timestamp, <int// in seconds, becomes the time when you become friends

            "name": "Xiaoming", <str// device alias,
            "phone": "13418100840", <str// device phone number

            "image": "http://XXX/XXX/XXX.jpg" <str// full URL image link
        },

        ....
```

```
  ]

}
```

Click "Delete friend" to send the result down -downDelFriend

```
{
    "success": 1, // 0 failure or 1 success
    "imei":"860748051983354",
    "type":"downDelFriend",
    "timestamp":1728627127086
}
```

When twatch is touched, it receives a message from a friend- downPPmessage

```
{

  "type"  : "downPPmessage",

  "messageurl": "" <string> //content [http://XXX](http://xxx/).amr or [http://

  "size": 1234, <integer>[*] // only when the image is available

  "duration": 15, <integer>[*] // only when speaking

  "friendimei": "123456789012345" <string> // friend's IMEI}
```

Device Long Sitting Reminder Set

```
{
        "sittingClockList": [
            {
        "disturbStatus": 1, //no disturb switch during lunch break //1: on 0: off
        "clockId": "jeuhvmth", //ID
        "name": "1111", //tag
        "startTime": "18:21", // start time
        "endTime": "18:21", // end time
        "switchStatus": 1, // sedentary reminder switch 1: on  0: off
            }
```

```
        ],
        "imei":    "" ,
        "type":
                "DEVICE_LONG_SITTING_REMINDE_SET"    ,
        "timestamp": 1743416476378
    }
```

Drink reminder CLOCK SET

```
{
        "imei": "",
        "waterClockList": [
            {
                "clockId": "dniptp84", //ID
                "name": "1444", // tag
                "startTime": "18:28", // start time
                "interval": 10, // Reminder interval in minutes
                "end time": "18:28", // end data
                "switchStatus": 1 //drinking reminder switch 1: on 0: off
            }
        ],
        "type": "DEVICE_WATER_CLOCK_SET",
        "timestamp": 1743416914810
    }
```

DownSchoolTimeTable

```
{
        "classTimesInfo": {[dict] // Course time period information Map [srt] // Key: No
            "1": "08:00-08:40",
            "2": "08:00-08:40",
            "3": "08:00-08:40",
            "4": "08:00-08:40",
            "5": "08:00-08:40",
            "6": "08:00-08:40",
            "7": "08:00-08:40",
            "8": "08:00-08:40"
        },
        "morningBreakTime": "08:40", [str] // morning break time
        "coursesInfoDict": {[dict] // specific course location and course information
            "1&1": {[dict] //key, Day of the week, Section, coordinates, value, course information
            (name)
                "name": "1111" // Name
            }


        },
        "imei":
            "" ,
        "type":
            "downSchoolTimeTable"      ,
        "timestamp": 1743418034993,
```

```
        "afternoonBreakTime": "08:40"          [str] // Afternoon recess time
    }
```

Down Watch Dial

```
{
        "imei": "xxxxx",
        "watchDialList": [//tableau {
         "filePath": "",// Dial file, download address
          "id": "23a47d14-0796-40a2-a60e-8edba3bea5d3" // Dial main key}
        ],
        "type": "downWatchDial",
        "timestamp": 1751349336757
    }
```

Star calendar- dnEphemeris

```
    {
        "datFilePath": "xxx.DAT", // Star Calendar number download file link
        "imei": "864688111111111",
        "type": "dnEphemeris",
        "timestamp": 1751351140365
    },
```

Set down Step Count Config

```
{
        "imei": "864688000000001",
        "periodList": [
            {
                "period": 1,
                "startTime": "16:17", // start time
                "endTime": "21:00", // end time
            },
            {
                "period": 2,
                "startTime": "14:01",
                "endTime": "17:09"
            },
            {
```

```
                "period": 3,
                "startTime": "06:14"
                "endTime": "11:08"          ,
            }

        ],
        "type": "downStepCountConfig",
        "timestamp": 1750065564606,
        "switch": 1 // Step setting switch, 1: on, 0: off

    }
```

Video call series instructions

Video process description

Pre-connection process:

Device-side initiative:

1. Video initiated by the device: The device uploads the upVideoCallWithAPP instruction, and the server will issue the downVideoCallWithAPPInfo instruction after processing the logic
2. Initialize the video SDK according to the content of downVideoCallWithAPPInfo instruction. After login, enter the corresponding video communication channel (video_id) and wait for the APP to receive or hang up
    1. Device active hang-up: Upload the upWatchHangUp command. Report the upVideoCallTime command to complete the entire video process.
    2. APP Hangup: The device receives the downAPPHangUp command. It reports the upVideoCallTime command to complete the entire video process.
    3. APP connection: The video SDK integrated with the device will receive the corresponding callback, and the video instruction process scheduling on the server is completed.

Initiated by the APP:

1. The device will receive the downVideoCallWithAPPInfo command
2. Initialize the video SDK and log in according to the content of downVideoCallWithAPPInfo instruction
    1. Device reception: Enter the corresponding video call channel (video_id) and complete the scheduling of video instruction process on the server.
    2. Device rejection: Upload the upWatchHangUp command. Report the upVideoCallTime command to complete the entire video process.

The process after connection:

1. APP Hangup: The device receives the down APP Hang Up command. It reports the up Video Call Time command to complete the entire video process.

2. Device hang-up: Report the up Video Call Time command to complete the whole video process.

Other exceptions:

Due to network card failure, or the video SDK does not receive video frames for a long time and other abnormal problems

Call the report up Video Call Time command to complete the entire video process.

Video series relationship profile ID description

Relationship image id is the number of the relationship profile picture

Default relationship name	The number of the relationship profile picture
father	1
mother	2
grandfather	3
grandmother	4
grandfather	5
grandmother	6
father's younger brother	7
aunt	8
user-defined	9

Up Get Video User List

Device upload:

```
{
    "imei":"352273017386001",
    "timestamp": 1648111390074,
    "type": "upGetVideoUserList"
}
```

Server reply:

```
{
    "imei":     "***"'
    "type":      "downGetVideoUserList"          ,

    "uuid_list": [
        {
            "relationship": "**", // Name of relationship
            "relationship_image_id": "", [int] relationship profile picture
            "phone": "", // mobile number
            "uuid": "",// user

        openID }
        ...
    ],
    "applet": <array> [// WeChat Mini Program user   video list



        {
            "appid"                          : <string// WeChat app ID
            "openid"                         : <String// WeChat Mini Program user ID
            "model_id"                       : <string> //Device model ID:
            "sn_ticket"                      :<string> //Device ticket
            "relationship"                   : <string> //Relationship name
            "relationship_image_id"          : <int> //Relationship avatar
            "connection_address"             : <string> // Video server connection address (IP: port)
            "appkey"                         :<string> // Vision's APPkey
        },.....
    ]





}
```

Up Video Call With APP Device upload:

```
{
    "imei": "***",
    "timestamp": 1648111390074, [],
    [millisecond]
    "type": "upVideoCallWithAPP",
    "user OpenID": "**", // The openID of the user to be called for video (exists to indicate calling to
    the APP)
    "call device": "**", // The IMEI of the device to be called (exists to indicate calling to the device)
    "call_type" *: 0, //[int] The type of the call (no such field: video) (0: video, 1: voice)}
```

Note that only one of user_openid and call_device can be stored at the same time

Server reply:

Note: The device needs to wait for down Video Call With APP Info to receive it and then call up the call interface

```
{
    "imei": "***",
    "type": "downVideoCallWithAPPInfo",
    "connection_address": "***", // video server connection address
    "appkey": "***", // the APPkey of the video terminal
    "username": "***", //, the account to which the
    device is logged in.
     "password": "***", //, the password to which
    the device is logged in
    "appUsername": "***", // the account to which the application is logged in
    "distinguishability_width": int, [int] The video resolution (width) of the device recording and
    transmitting to the mobile terminal.
     "distinguishability_heiger": int, [int] The video resolution (height) of the device recording and
    transmitting to the mobile terminal
    "video_id": "***", // The ID of the call
    "limit_time": int, // Limit the duration of video call (unit: s)
    "wait_time": int, // Waiting time to enter the channel (unit: s)
     "contact": "",// relationship name
    "relationship": "", // Name of relationship
    "relationship_image_id": int, // [int] relationship profile picture
    "call_type": int, // [int] call type 0-video,1-voice
    "deviceVideoType": "VM", //SW (voice network) or JF (Jiufeng) type, or VM (Shitong)}
```

The device has not been connected and hung up after initiating a video call - UpWatch Hang Up

Device upload:

```
{
    "imei": "***",
    "timestamp": 1648111390074, [millisecond]
    "type": "upWatchHangUp",
    "video_id": "***" //[str] The ID of the video call this time
    "call_mode" *: <int> //1=app call (default is not passed) 2=wechat app call}
```

APP/DEVICE Hang up after initiating a video/voice call (when the device has not received a ring.) Down/or connected) - downAPPHang

Up server sends:

```
{
    "imei": "***",
    "type": "downAPPHangUp",
    "timestamp": 1648111390074, [] [millisecond]
    "video_id": "***" // [str] corresponds to the video_id of the hang-up
```

Device upload video call duration-upVideoCallTime

Device upload:

```
{
    "imei": "***",
    "timestamp": 1648111390074, [],
    milliseconds "type": "upVideoCallTime",
    "video_id": "***", // the ID of the video call this time
    "duration": int, // [int] video duration If not connected, duration: 0}
```

Upload the network status of the device during the video call - upVideoNetStatusWithAPP

Device upload:

```
{
    "imei": "***",
    "timestamp": 1648111390074, [],
    [millisecond] "type":
    "upVideoNetStatusWithAPP",
    "video_id": "***", // The ID of the video call this time
    "status": 3, // Network status: 1: good; 2: general; 3: poor; 4: very poor
}
```

APP/device Invite device video/voice call -downVideoCallWithWatch

Server sends:

```
{
    "imei": "***",
    "type": "downVideoCallWithWatch",
    "connection_address": "***", // video server connection address
    "appkey": "***", // initialization APPkey of video SDK
    "username": "***", //, the account to which the device
    wants to log in.
    "password": "***", //, the password to which the
    device wants to log in
    "app_username": "***", // the account to which the app is logged in
    "distinguishability_width"  : int, //[int] device recording video resolution width
    "distinguishability_height": int, [int] device recording video resolution height
    "video_id": "***", // this call ID; if it's a WeChat Mini Program video: WeChat Room ID for this call
    "limit_time": int, // video call duration limit (unit: s)
     "wait_time": int, // waiting time to enter the channel(unit: s)
     "contact": "***", // Name of the relationship
```

```
        "relationship": "***", // Name of relationship
        "relationship_image_id": int, // [int] relationship profile picture
        "call_type": 0, // [int] call type 0-video, 1-voice
        "deviceVideoType": "VM", //SW (voice network) or JF (Jiufeng) type, or VM (Shitong)

        //===== WeChat Mini program call and release =====
        "appid" : <string>    //wx_appid
        "call_mode": <int> //1=app call   2=wechat app call
        "sn_ticket": <string> // Device ticket (valid for minutes)
        "payload": <string> // Custom paylo sent by a third party when initiating a call
        "model_id": <string> // Device model ID registered in WeChat

        //===== WeChat Mini program call release-end=====

}
```

Down Video Call Fail Info

Server reply:

```
{
        "imei": "***",
        "type": "downVideoCallFailInfo",
        "message": "" // Cause of failure}
```

Micro chat series of instructions

Up Get Chat User List

Device upload:

```
{
        "type": "upGetChatUserList",
        "imei": "860256954789522",
        "timestamp": 1692151451167
}
```

Server reply:

```
{
        "imei":"860256954789522",
```

```
    "type":"downGetChatUserList",
    "timestamp":1692151451834,
    "groupList":
    [
        {
      "groupId": <string>, //Group ID (when groupType==2, it is...
      "groupType": <int> // mode 0: group chat 1: single chat 2: bump
      "relationship_image_id" *: <int>, // The profile picture ID is only available in one-on-one
chats
      "group Name": <string>, // Group Name (single chat: relationship name, group chat:
      "group Image": <string>, // The URL of the group profile picture. If not, it is empty by default
      "createGroupTime": <Long>, // group creation timestamp in milliseconds
      "callOpenid":* <string>, // The openID of a user who can be called, only for single chat
      "canVideo": <int> // Whether it supports call (video/voice) 0:
        }
    ]





}
```

Chat new message reminder-downChatVoice

Messages received from the APP, for example, are uploaded by the device using an interface in the HTTP protocol.

Server sends:

```
{
    "type": "downChatVoice"
    "ident": <int> //instruction identifier
    "openid": <String>Sender user ID
    "imei": <String>//IMEI
   "groupId": <string>// Group ID (when groupType==2
    "groupType": <int> // mode 0: group chat 1: single chat 2: kengkeng
    "id": <string// message ID
    "senderAvatar": <String>//Sender avatar
    "relationship": <string// relationship name (when groupType==2
   "relationship_image_id": <int> // relationship header (refer to the video series relationship header
    "content type": <int> //1: text 2: picture 3: voice
    "url": <string>// The download address of the voice/image file
    "duration" *: <float> // Duration of voice/image/file playback
    "size": <int> // The byte size of the voice/image file
    "content": <string> exists when content Type=1
    "bqId": <string>// exists whencontent Type=4
    " mute": <int> // Whether the message is muted
   "timestamp": <long> // Time stamp
```

Device reply:

```
{
    "type": "downChatVoice",
    "imei": <String>, //IMEI
    "timestamp": <long> //timestamp

}
```

JSON contains uniform validation parameters

Note: JSON packets that do not conform to this rule instruction are considered invalid packets and can be discarded

JSON contains a uniform validation parameter:

EncryptionCode: Encryption verification parameter

Rule：

For background and device development, agree on a Key: TestKey

JSON data parameters to be transmitted are sorted in ascending order (dictionary order) by the ASCII code of the field name. After sorting, this is followed by one Key piece

Perform MD5 encryption to get a 32-bit uppercase Code

==== Attention!====

Encryption of JSON requires multiple nesting. Refer to the encryption code for details

The code is added to the json, and the data is (encryptionCode: Code) to form the final JSON transmission value

eg:

```
Key:TestKey
json:{"type":1,"ident":654897,"imei":"666666666666666","imsi":"946002686475

 montage：
heartbeat:50:ident:654897:imei:666666666666666:imsi:9460026864759229:module

The 32-bit uppercase Code after MD5 is: 1A6B272777C3C78FC5BB8C09A237767F

 final JSON：
{"type":1,"ident":654897,"imei":"666666666666666","imsi":"9460026864759229
```

Example of encrypted JAVA code:

```
import com.alibaba.fastjson.JSONArray;
import com.alibaba.fastjson.JSONObject;
        /**
         * Generate a verification signature
         * @param jsonStr
         * @param KEY
         * @return
         */
    public static String getSign(String jsonStr, String KEY) {

        String result = "";
        try {
            JSONObject map = JSONObject.parseObject(jsonStr);
            result = getJSONObjectSortString(map);
            // Finally, it is spliced with Key and encrypted by MD5
            result = DigestUtils.md5Hex(result+KEY).toUpperCase();
        } catch (Exception e) {
            return null;
        }
        return result;
    }

    private static String getJSONObjectSortString(JSONObject map){
        List<Map.Entry<String, Object>> infoIds = new ArrayList<Map.Entry<S
        // All incoming parameters are sorted in ascending order by the ASCII code of the field name
        (dictionary order)
        Collections.sort(infoIds, new Comparator<Map.Entry<String, Object>>
            public int compare(Map.Entry<String, Object> o1, Map.Entry<Stri
                return (o1.getKey()).compareTo(o2.getKey());
            }
        });
        // Format the signature key pair
        StringBuilder sb = new StringBuilder();
        for (Map.Entry<String, Object> item : infoIds) {
            if (item.getKey() != null || item.getKey() != "") {
                String key = item.getKey();
                Object val = item.getValue();
                if (val != null) {
                    if(val instanceof JSONObject){
                        val = getJSONObjectSortString((JSONObject)val);
                    }
                    if(val instanceof JSONArray){
                        val = getJSONArraySortString((JSONArray) val);
                    }
                    sb.append(key + ":" + val + ":");
                }
            }

        }
        return sb.toString();
    }

    private static String getJSONArraySortString (JSONArray list){
```

```
        // list concatenation
        StringBuilder sb = new StringBuilder();
        for (Object value: list) {
            if(value != "") {
                if(value instanceof JSONObject){
                    value = getJSONObjectSortString((JSONObject)value);
                }
                if(value instanceof JSONArray){
                    value = getJSONArraySortString((JSONArray) value);
                }
                sb.append(value + ":");
            }
        }
        return sb.toString();




    }
```

An encrypted Python example:

```
# Introduction to the official document: https://docs.python.org/3/library/hashlib.html?highlight=ha
# Use md5 encoding library
import hashlib
import json

def getSign(jsonStr, coderKey):
    """
    To validate the JSON string and key,
    generate a signature
    :param jsonStr:
    :param coderKey:
    :return:
    """
    jsonData = json.loads(jsonStr)
    tempStr = getDictSortString(jsonData)
    sign = hashlib.md5(tempStr + coderKey).hexdigest().upper()
    return sign


def getDictSortString(jsonData):
    jsonDataKeys = jsonData.keys()
    # print "Before sorting: Data keys:%s"%jsonDataKeys
    s_jsonDataKeys = sorted(jsonDataKeys)
    # print "After sorting: s_jsonDataKeys:%s"% s_jsonDataKeys
    tempStr = ""
    for key in s_jsonDataKeys:
        value = jsonData.get(key)
        if value is not None:
            if isinstance(value, dict):
                value = getDictSortString(value)
            if isinstance(value, list):
                value = getListSortString(value)
            tempStr += "%s:%s:" % (key, value)
    return tempStr
```

```
def getListSortString(jsonList):
    tempAllStr = ""
    for value in jsonList:
        if value is not None:
            tempStr = value
            if isinstance(value, dict):
                tempStr = getDictSortString(value)
            if isinstance(value, list):
                tempStr = getListSortString(value)
            tempAllStr += "%s:" % tempStr
    return tempAllStr
```

Device-side HTTP interaction protocol

If you need the device to support HTTPS, you can replace it with an HTTPS interface

Communication address of the device platform
Contact the backend developer for specific domain names and ports

Default port: 80
HTTP:

```
XXX.XXX.XXX
```

Signature method for Sign parameter:

```
Param parameters:

Key: The development Key previously agreed upon by TestKey

The JSONdata parameters transmitted are sorted in ascending order by the ASCII code of the field name (in dictionary order) and then concatenated to this Key

    Perform MD5 encryption to get a 32-bit uppercase Code

----The algorithm refers to the above algorithm that contains unified verification parameters in the JSON----

The content of the uploaded file is not encrypted
```

The server returns success to indicate that the call interface has been uploaded successfully

·500 Contact the background developer

·400 The device is not online (you need to log in again)

·401 The signature is incorrect
·402 Parameter error

·403 The user does not exist (the user list needs •to be retrieved again)

·404 Check the path
·200 and return content is success is normal

Send GroupMSG-Chat messages to APP

url:

```
http://XXX.XXX.XXX/send/groupmsg
```

method:

```
POST
```

content-type:

```
multipart/form-data
```

params:

```
imei: <string> // Device imei parameter
Signature: <string> // The verified signature
timestamp: <long> // Time stamp
groupId: <string> // Group ID sent (when groupType==2, it is for the friend on the other end
group Type: <int> // Mode 0: Group chat 1: Single chat 2: Bump for friendship chat
content*: <file> // Data
contentType: <int> // Default: 0 0: voice 1: picture 2:expression
bqId*: <string> //expressionID (contentType: 2 only when available)
size*: <int> // Data size
audioDuration*: <int> // Voice duration (Only available when content type: 0)
(to match the voice on the APP)
ident: <int> // 6-bit random positive integer
audioType: <string> //Audio format, e.g. aac, amr...
```

result:

```
success
```

Send/RemoteImage -Remote photos sent to the APP

url:

```
http://XXX.XXX.XXX/send/remoteimage
```

method:

```
POST
```

content-type:

```
multipart/form-data
```

params:

```
imei: <string> // Device imei parameter
signature: <string> // The verified signature
timestamp: <long> // Timestamp
content*: <file> // Data
size*: <int> // Data size
ident: <int> // 6-bit random positive integer
```

result:

```
success
```